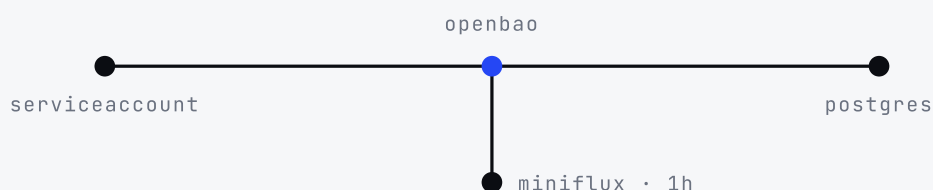


DYNAMISCHE DATENBANK CREDENTIALS

Kurzlebige PostgreSQL-Zugänge mit OpenBao und dem External Secrets Operator auf Kubernetes.



Dynamische Datenbank-Credentials mit OpenBao

PostgreSQL-Anbindung über den Database Secrets Engine und External Secrets Operator – am Beispiel Miniflux auf Kubernetes

Thomas Zachmann

17. September 2026

Inhalt

Worum es geht

- Warum von Hand, und warum ohne KI
- Ein Wort zu den Passwörtern in diesem Leitfaden
- Für wen
- Die Bausteine
- Die Architektur in einem Bild

Teil I – Der Cluster

- OpenBao im Cluster
- PostgreSQL per Helm
- External Secrets Operator per Helm
- Namespace und ServiceAccount der Anwendung

Teil II – OpenBao einrichten

- Kubernetes-Auth: Wer darf sich anmelden?
- Database Secrets Engine: Die Verbindung
- Database Secrets Engine: Die Rollen
- Policy: Was darf der angemeldete Token?

Teil III – PostgreSQL vorbereiten

- Die Rolle, mit der OpenBao arbeitet
- Die Owner-Rolle für Miniflux
- Verifikation: Läuft die Session wirklich als miniflux?

Teil IV – External Secrets Operator anbinden

- SecretStore: Wie ESO zu OpenBao kommt
- VaultDynamicSecret: Der Generator
- ExternalSecret: Das Ziel-Secret
- Anwenden und prüfen

Teil V – Die Anwendung

- Das Deployment
- Die Oberfläche ansehen

Teil VI – Was schiefging, und warum

- permission denied for schema public
- "miniflux" is not an allowed role
- permission denied (403) bei allem
- permission denied to grant role "miniflux"
- Logs des falschen Pods

Teil VII – Betrieb

- Die TTL-Falle: Env-Variablen sind statisch
- Root-Token ablösen

Infrastructure as Code

Klartext-Passwörter

Lease-Hygiene

Anhang A – Alle Kommandos auf einen Blick

Anhang B – Glossar

Worum es geht

Eine Anwendung braucht Zugang zu einer Datenbank. Der klassische Weg: ein Benutzer mit festem Passwort, das irgendwo in einem Kubernetes-Secret liegt – oft für Monate oder Jahre, kopiert in Helm-Values, CI-Variablen und Notizbücher. Wird es kompromittiert, merkt das niemand, und die Rotation ist so mühsam, dass sie unterbleibt.

Dieser Leitfaden beschreibt die Alternative: **Die Anwendung bekommt gar kein festes Passwort mehr.** Stattdessen erzeugt OpenBao auf Anfrage einen Datenbank-Benutzer mit kurzer Lebensdauer (hier: eine Stunde), der External Secrets Operator (ESO) legt die Zugangsdaten als Kubernetes-Secret ab, und die Anwendung liest sie wie gewohnt aus einer Umgebungsvariable. Läuft die Lebensdauer ab, löscht OpenBao den Benutzer wieder; ESO hat längst neue Zugangsdaten geholt.

Der rote Faden ist ein konkretes, nachvollziehbares Beispiel: **Miniflux**, ein schlanker RSS-Reader in Go, der eine PostgreSQL-Datenbank braucht und beim Start seine eigenen Schema-Migrationen ausführt. Genau dieses Detail – die App legt selbst Tabellen an – macht das Beispiel lehrreich, denn es zwingt uns, über Eigentümerschaft von Datenbankobjekten nachzudenken.

Alles, was hier steht, wurde in einem RKE2-Homelab-Cluster tatsächlich durchgeführt. Die Fehler, die dabei aufgetreten sind, stehen mit drin – sie sind der wertvollste Teil.

Es ist der zweite Teil einer Reihe. N° 1, *OpenBao auf Kubernetes*, baut den OpenBao, der hier benutzt wird – Helm, init und unseal, Kubernetes-Auth. N° 3 beschreibt denselben OpenBao auf einer VM, mit Ansible, OpenTofu und Disaster Recovery. Dieser Leitfaden setzt N° 1 voraus und wiederholt davon nur, was für den Zusammenhang nötig ist.

Warum von Hand, und warum ohne KI

In einem echten Setup baut man das nicht per CLI. OpenBao-Rollen, Policies, Auth-Methoden und Datenbank-Connections gehören in HashiCorp Terraform oder OpenTofu: versioniert, reproduzierbar, im Review nachvollziehbar. Der Anhang nennt die passenden Ressourcen, und das Repository, aus dem dieser Leitfaden entstanden ist, verwaltet Auth und Policies bereits so.

Trotzdem führt dieser Leitfaden jeden Schritt einmal von Hand aus – und zwar bewusst ohne KI-Assistenz als Abkürzung. Ein Terraform-Modul, das jemand anderes oder ein Sprachmodell geschrieben hat, versteckt genau das, was man verstanden haben muss, wenn es um drei Uhr nachts nicht funktioniert: welche Komponenten beteiligt sind, wer sich bei wem ausweist, wer welche Rechte braucht und in welcher Reihenfolge der Fluss läuft.

Der Maßstab ist einfach. Wer diesen Leitfaden durchgearbeitet hat, sollte die Architektur aus dem nächsten Abschnitt **ohne Hilfe auf ein leeres Blatt zeichnen** können: die vier Komponenten, die sechs Schritte, die zwei Schranken (Allowlist der Connection und Policy) und die eine PostgreSQL-Rolle, der alles gehört. Wer das kann, kann das Terraform-Modul auch schreiben – oder beurteilen, ob das, was eine KI vorschlägt, richtig ist.

Ein Wort zu den Passwörtern in diesem Leitfaden

In diesem Leitfaden stehen Passwörter im Klartext: postgres123, demo123, admin123, dazu Usernames dynamischer Datenbank-Benutzer. Das ist kein Versehen. Alles hier stammt aus einer **Entwicklungsumgebung** – ein Homelab-Cluster ohne Zugang von außen, ohne echte Daten, mit einer Datenbank, die jederzeit weggeworfen und neu aufgesetzt werden kann. Die Werte sind gewählt, damit man sie beim Lesen wiedererkennt, nicht damit sie etwas schützen.

In jeder anderen Umgebung gilt das Gegenteil: Kein Passwort in Helm-Values, keines in einem Deployment-Manifest, keines in einer Dokumentation. Genau dafür gibt es das, was hier beschrieben wird – die Anwendung bekommt gar kein festes Passwort mehr, und die wenigen statischen Geheimnisse, die bleiben (Root-Credentials für die Datenbank, das Miniflux-Admin-Passwort), gehören in das KV-Engine von OpenBao und von dort per ExternalSecret in den Cluster. Teil VII geht darauf ein.

Für wen

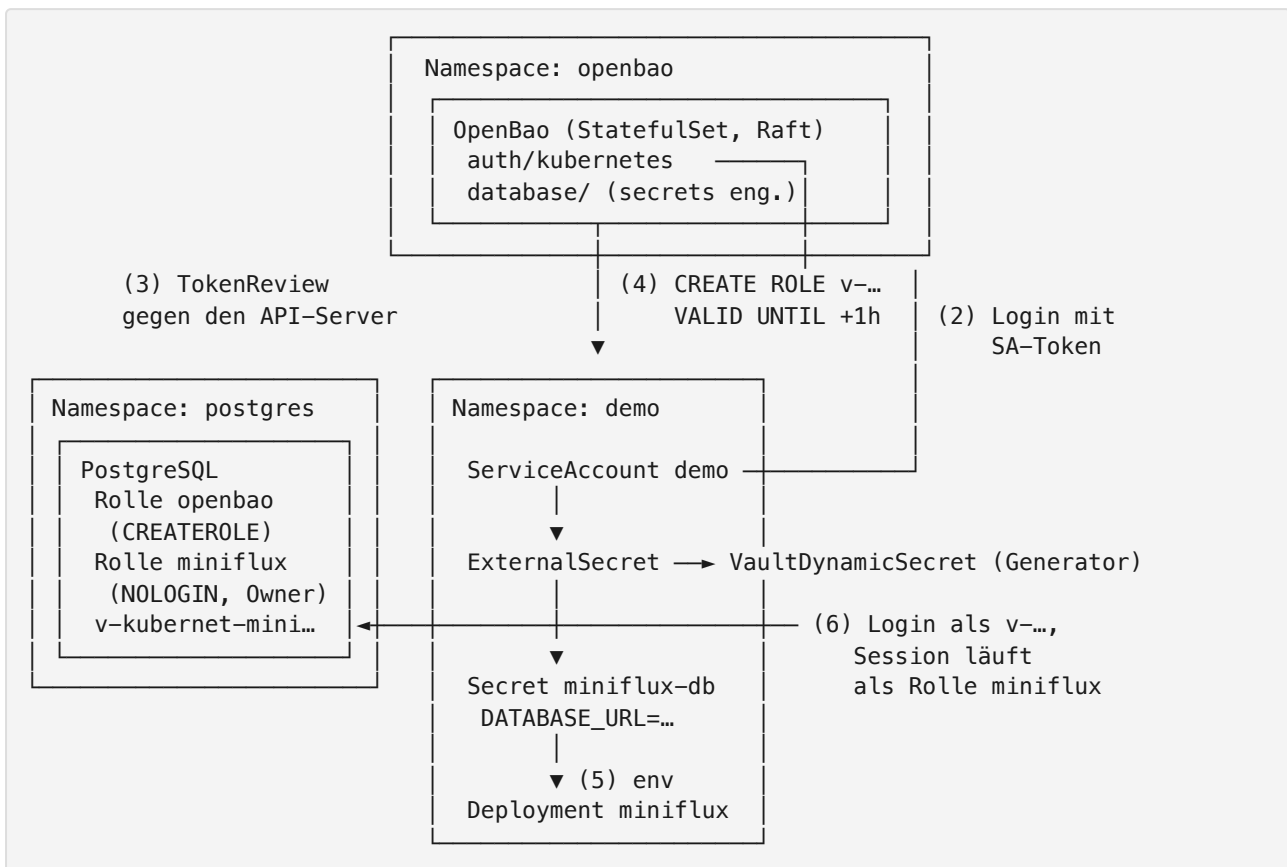
Für Leserinnen und Leser, die Kubernetes-Grundlagen (Deployment, Secret, ServiceAccount, Helm) kennen, PostgreSQL bedienen können und OpenBao bzw. HashiCorp Vault schon einmal gesehen haben. OpenBao ist ein Fork von Vault; alles hier gilt für beide, lediglich der CLI-Name (bao statt vault) und die Umgebungsvariablen (BAO_ADDR statt VAULT_ADDR) unterscheiden sich.

Dieser Leitfaden ist kostenlos und bewusst kurz: ein einziger Anwendungsfall, zu Ende gedacht bis zur laufenden Anwendung. Er erklärt, was er braucht, aber er ersetzt keine Einführung. Wer Auth-Methoden, Policies, Secrets Engines und Leases von Grund auf verstehen will – mit Labs, die auf dem Laptop laufen –, findet das in meinem Buch **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice): 24 Kapitel, darunter der Database Secrets Engine und die Kubernetes-Auth-Methode; OpenBao kommt in Kapitel 18.

Die Bausteine

Baustein	Version	Rolle im Zusammenspiel
RKE2 Kubernetes	–	Laufzeitumgebung, Traefik als Ingress
OpenBao (Helm-Chart openbao/openbao)	Chart 0.29.4, App 2.6.2	Erzeugt und widerruft Datenbank-Benutzer; authentifiziert Workloads
PostgreSQL (Helm-Chart bitnami/postgresql)	Chart 18.11.3, PG 18.6	Die Datenbank
External Secrets Operator	2.10.0	Holt Secrets aus OpenBao und schreibt Kubernetes-Secrets
Miniflux	2.3.3	Die Beispielanwendung

Die Architektur in einem Bild



Der Ablauf, der in diesem Bild steckt:

1. ESO reconciled das ExternalSecret und stößt den Generator VaultDynamicSecret an.
2. Der Generator meldet sich bei OpenBao an – mit dem Token des ServiceAccounts demo (Kubernetes-Auth-Methode).
3. OpenBao prüft das Token per TokenReview gegen den Kubernetes-API-Server und stellt einen OpenBao-Token mit der Policy eso-demo aus.
4. Mit diesem Token liest der Generator database/creds/miniflux. OpenBao verbindet sich als Rolle openbao zur Datenbank und führt die creation_statements aus: ein neuer Login-User v-kubernetes-miniflux-... entsteht, gültig für eine Stunde.
5. ESO rendert daraus per Template eine DATABASE_URL und schreibt sie in das Kubernetes-Secret miniflux-db. Das Deployment liest sie als Umgebungsvariable.
6. Miniflux verbindet sich. Dank SET ROLE läuft die Session sofort als miniflux – alle Tabellen, die die Migration anlegt, gehören dieser festen Rolle, nicht dem kurzlebigen User.

Die Kapitel folgen diesem Weg: erst die Infrastruktur im Cluster, dann OpenBao, dann PostgreSQL, dann ESO, dann die Anwendung. Am Ende stehen die Fehler, die unterwegs auftraten, und was im Betrieb zu beachten ist.

Teil I – Der Cluster

OpenBao im Cluster

OpenBao läuft als StatefulSet im Namespace openbao, mit Raft-Storage auf einem PVC, standalone, manuell initialisiert und entsiegelt. Wie es dorthin kommt – Helm-Values, init, unseal, Ingress – ist Inhalt von N° 1. Für diesen Leitfaden zählen drei Dinge:

- Der In-Cluster-Service heißt openbao.openbao.svc:8200; das ist die Adresse, die ESO benutzt.
- Die Kubernetes-Auth-Methode ist aktiviert und konfiguriert (auth/kubernetes/config zeigt auf <https://kubernetes.default.svc:443>).
- Die Einrichtung läuft in einer Shell im Pod, weil dort BA0_ADDR=http://127.0.0.1:8200 gesetzt ist:

```
kubectl exec -it -n openbao openbao-0 -- sh
/ $ bao login          # Root-Token eingeben
```

Alle bao-Kommandos in diesem Leitfaden sind in einer solchen Shell ausgeführt worden. Ein häufiger Stolperstein: Ein Terminal mit BA0_ADDR auf den Ingress, aber ohne gültigen Token, liefert für *jeden* Aufruf 403 permission denied – das sieht aus wie ein Policy-Fehler, ist aber schlicht ein fehlendes Login.

PostgreSQL per Helm

Das Bitnami-Chart, minimal konfiguriert (postgres/values.yaml):

```
auth:
  postgresPassword: "postgres123"  # Superuser – nur für die Demo im Klartext
  username: "demo"
  password: "demo123"
  database: "demodb"

primary:
  persistence:
    enabled: true
    size: 10Gi
```

```
helm upgrade --install postgres oci://registry-1.docker.io/bitnamicharts/postgresql \
--namespace postgres --create-namespace \
--values postgres/values.yaml
```

Ergebnis: ein Pod postgres-postgresql-0, Service postgres-postgresql.postgres.svc.cluster.local:5432, Datenbank demodb.

Für Admin-Arbeiten am schnellsten direkt im Pod. Das Bitnami-Image legt das Superuser-Passwort als Datei ab, daher der cat:

```
kubectl exec -it -n postgres postgres-postgresql-0 -- sh -c \
  'PGPASSWORD="$(cat /opt/bitnami/postgresql/secrets/postgres-password)" \
  psql -U postgres -d demodb'
```

Es empfiehlt sich, das als kleine Shell-Funktion oder Alias abzulegen; es wird noch einige Male gebraucht.

External Secrets Operator per Helm

```
helm repo add external-secrets https://charts.external-secrets.io
helm upgrade --install external-secrets external-secrets/external-secrets \
  --version 2.10.0 \
  --namespace external-secrets --create-namespace
```

ESO bringt die CRDs SecretStore, ExternalSecret und – wichtig für dieses Leitfaden – die **Generatoren** mit, darunter VaultDynamicSecret. Ein Generator ist ein Objekt, das bei jedem Sync ein *neues* Secret erzeugt, statt ein bestehendes zu lesen. Genau das brauchen wir für dynamische Credentials.

Namespace und ServiceAccount der Anwendung

```
kubectl create namespace demo
kubectl create serviceaccount demo -n demo
```

Dieser ServiceAccount ist die **Identität**, mit der sich ESO bei OpenBao anmeldet. Er braucht keine Kubernetes-RBAC-Rechte; OpenBao prüft lediglich, dass ein Token für genau diesen Account in genau diesem Namespace vorliegt.

Teil II – OpenBao einrichten

Alle Kommandos in diesem Teil laufen mit dem Root-Token in der Pod-Shell (siehe Teil I). In einem realen Setup gehört das nach der Ersteinrichtung in Terraform/OpenTofu – dazu mehr in Teil VII.

Kubernetes-Auth: Wer darf sich anmelden?

Die Auth-Methode kubernetes erlaubt Workloads, sich mit ihrem ServiceAccount-Token anzumelden. OpenBao reicht das Token zur Prüfung an den API-Server weiter (TokenReview).

```
bao auth enable kubernetes

bao write auth/kubernetes/config \
  kubernetes_host="https://kubernetes.default.svc:443"
```

Läuft OpenBao selbst im Cluster, reicht das: Es benutzt sein eigenes Pod-Token und die eingebaute CA, um den TokenReview durchzuführen. Läuft OpenBao außerhalb, müssen `kubernetes_ca_cert` und `token_reviewer_jwt` zusätzlich gesetzt werden.

Dann die **Rolle**, die einen ServiceAccount auf Policies abbildet:

```
bao write auth/kubernetes/role/eso-demo \
  bound_service_account_names=demo \
  bound_service_account_namespaces=demo \
  token_policies=eso-demo \
  token_ttl=1h
```

Kontrolle:

```
/ $ bao read auth/kubernetes/role/eso-demo
Key                                Value
---                                -
alias_name_source                  serviceaccount_uid
bound_service_account_names        [demo]
bound_service_account_namespaces   [demo]
token_policies                     [eso-demo]
token_ttl                          1h
```

Merke: Die Rolle sagt *wer* (SA demo in Namespace demo) und *mit welchen Rechten* (Policy eso-demo). Was die Rechte konkret erlauben, steht in der Policy – das ist der nächste Schritt nach dem Secrets Engine.

Database Secrets Engine: Die Verbindung

```
bao secrets enable database
```

Eine **Connection** beschreibt, wie OpenBao sich zur Datenbank verbindet, und – wichtig – welche Rollen diese Verbindung benutzen dürfen:

```

bao write database/config/postgres-demo \
  plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@postgres-postgresql.postgres.svc.
    cluster.local:5432/demodb?sslmode=disable" \
  username="openbao" \
  password="<Passwort der PG-Rolle openbao>" \
  password_authentication="scram-sha-256" \
  allowed_roles="demo-app"

```

Die PG-Rolle openbao muss vorher existieren (Teil III). Die Platzhalter `{{username}}` und `{{password}}` werden von OpenBao selbst gefüllt; das hat den Vorteil, dass OpenBao das Passwort später mit `bao write -f database/rotate-root/postgres-demo` rotieren kann, ohne dass es noch jemand kennt.

`allowed_roles` ist eine Allowlist. Nur die hier genannten Datenbank-Rollen dürfen über diese Connection Credentials erzeugen. Das ist die erste von zwei Schranken, die später beim Anlegen der Miniflux-Rolle im Weg standen.

So sieht die Connection nach der Einrichtung aus (das Passwort zeigt OpenBao nie wieder an):

```

/ $ bao read database/config/postgres-demo
Key                               Value
---                               -
allowed_roles                     [demo-app]
connection_details                map[connection_url:postgresql://{{username}}:{{password}}@postgres-
postgresql.postgres.svc.cluster.local:5432/demodb?sslmode=disable password_authentication:
scram-sha-256 username:openbao]
plugin_name                       postgresql-database-plugin

```

Database Secrets Engine: Die Rollen

Eine **Rolle** im Database Engine ist eine Vorlage: Welche SQL-Statements werden ausgeführt, wenn jemand Credentials anfordert, und wie lange leben sie?

Die erste Rolle: demo-app (nur Lesen und Schreiben)

Diese Rolle existierte bereits für eine andere Demo-App. Sie ist ein gutes Beispiel für *Least Privilege* auf bestehende Daten:

```

bao write database/roles/demo-app \
  db_name=postgres-demo \
  default_ttl=1h \
  max_ttl=24h \
  creation_statements='
    CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '""{{password}}""' VALID UNTIL '""{{e
      xpiration}}""';
    GRANT CONNECT ON DATABASE demodb TO "{{name}}";
    GRANT USAGE ON SCHEMA public TO "{{name}}";
    GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO "{{name}}";
    GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO "{{name}}";'

```

(Die ''''-Konstruktion ist die POSIX-Art, ein einfaches Anführungszeichen innerhalb eines einfach-gequoteten Strings zu setzen. Alternativ die Statements in eine Datei schreiben und `creation_statements=@file.sql` übergeben.)

Die Platzhalter `{{name}}`, `{{password}}` und `{{expiration}}` füllt OpenBao bei jedem Aufruf. `{{name}}` folgt dem Muster `v-<auth>-<rolle>-<random>-<ts>`, also z. B. `v-kubernetes-demo-app-ALeDiNgGWCikBp74UKVD-1789630487`. Der zweite Teil verrät, mit welcher Auth-Methode die Credentials angefordert wurden – `kubernetes` für Kubernetes-Auth, `root` für den Root-Token. Das ist beim Debuggen erstaunlich nützlich.

Warum diese Rolle für Miniflux nicht taugt – und das ist der zentrale Lerneffekt des Leitfadens:

1. Es fehlt `CREATE` auf dem Schema. Seit PostgreSQL 15 hat die Pseudo-Rolle `PUBLIC` per Default kein `CREATE` auf `public` mehr. Miniflux kann also keine Tabellen anlegen.
2. `GRANT ... ON ALL TABLES IN SCHEMA public` ist ein **Snapshot**. Es wirkt nur auf Tabellen, die zum Zeitpunkt des Grants existieren. Bei einer leeren Datenbank grantet es auf nichts. Und die Tabellen, die Miniflux dann anlegt, gehören dem dynamischen User – nach einer Stunde ist der weg und sein Nachfolger hat keine Rechte darauf.

Die zweite Rolle: `miniflux` (Owner-Role-Muster)

Die Lösung: Eine **feste, nicht anmeldbare Rolle** besitzt das Schema und alle Objekte. Jeder dynamische User wird Mitglied dieser Rolle und schlüpft bei jeder Session automatisch hinein.

```
bao write database/roles/miniflux \
  db_name=postgres-demo \
  default_ttl=1h \
  max_ttl=24h \
  creation_statements='CREATE ROLE "{{name}}" WITH LOGIN PASSWORD ''''{{password}}'''' V
                        ALID UNTIL ''''{{expiration}}'''' IN ROLE miniflux; ALTER ROLE "{{name}}" SET R
                        OLE miniflux;'
```

Zwei Statements, zwei Aufgaben:

- `... IN ROLE miniflux` macht den neuen User zum Mitglied der Rolle `miniflux`. Er *darf* damit alles, was `miniflux` darf.
- `ALTER ROLE "{{name}}" SET ROLE miniflux` setzt die Default-Rolle für jede Session dieses Users. Ohne diese Zeile müsste die Anwendung selbst `SET ROLE miniflux` absetzen – Miniflux tut das nicht, und die meisten Anwendungen ebenso wenig. Mit dieser Zeile *ist* jede Verbindung sofort `miniflux`, und alles, was sie anlegt, gehört `miniflux`.

Nebeneffekt: Weil der dynamische User selbst nichts besitzt, klappt das `DROP ROLE` bei der Revocation immer. Beim naiven Ansatz scheitert es, sobald der User Tabellen angelegt hat (`role cannot be dropped because some objects depend on it`).

Die Connection um die neue Rolle erweitern

Das war der erste Fehler, der auftrat:

```
/ $ bao read database/creds/miniflux
Error reading database/creds/miniflux: ... Code: 500. Errors:
* "miniflux" is not an allowed role
```

Die Connection kennt nur demo-app. Ein write auf die Config ist ein *Merge* – bestehende Felder bleiben erhalten, nur die übergebenen werden ersetzt (OpenBao prüft dabei kurz die DB-Verbindung):

```
bao write database/config/postgres-demo allowed_roles="demo-app,miniflux"
```

Für eine Demo wäre `allowed_roles="*"` bequemer. Die explizite Liste ist der sauberere Weg: Sie verhindert, dass jemand mit Schreibrechten auf `database/roles/*` sich über eine fremde Connection Credentials verschafft.

Policy: Was darf der angemeldete Token?

Die Policy `eso-demo` ist bewusst explizit – pro Pfad genau eine Capability:

```
path "secret/data/demo/demo-app" {
  capabilities = ["read"]
}

path "secret/metadata/demo/demo-app" {
  capabilities = ["read"]
}

path "database/creds/demo-app" {
  capabilities = ["read"]
}

path "database/creds/miniflux" {
  capabilities = ["read"]
}
```

`bao policy write` ersetzt die Policy vollständig, also immer den ganzen Text übergeben:

```
bao policy write eso-demo - <<'EOF'
path "secret/data/demo/demo-app"      { capabilities = ["read"] }
path "secret/metadata/demo/demo-app"  { capabilities = ["read"] }
path "database/creds/demo-app"        { capabilities = ["read"] }
path "database/creds/miniflux"        { capabilities = ["read"] }
EOF
```

Policies werden bei *jedem Request* ausgewertet, nicht beim Login. Bereits ausgegebene ESO-Tokens profitieren also sofort; ein Neustart von ESO ist nicht nötig.

Damit sind die zwei Schranken benannt, die beide passen müssen, bevor Credentials fließen:

Schranke	Wo	Fehlermeldung, wenn sie fehlt
allowed_roles	database/config/<connection>	"miniflux" is not an allowed role (HTTP 500)
Policy-Pfad	sys/policies/acl/<policy>	permission denied (HTTP 403)

Teil III – PostgreSQL vorbereiten

Alle Statements als Superuser postgres in demodb (siehe Teil I für den psql-Einstieg).

Die Rolle, mit der OpenBao arbeitet

OpenBao braucht einen eigenen Login-User, der andere Rollen anlegen und löschen darf. Kein Superuser – CREATEROLE reicht:

```
CREATE ROLE openbao WITH LOGIN PASSWORD '...' CREATEROLE;
```

Dieses Passwort landet einmalig in database/config/postgres-demo (Teil II) und kann danach von OpenBao rotiert werden.

Die Owner-Rolle für Miniflux

```
CREATE ROLE miniflux NOLOGIN;  
GRANT CONNECT ON DATABASE demodb TO miniflux;  
GRANT USAGE, CREATE ON SCHEMA public TO miniflux;  
GRANT miniflux TO openbao WITH ADMIN OPTION;
```

Die letzte Zeile ist seit **PostgreSQL 16** zwingend und leicht zu übersehen: CREATE ROLE ... IN ROLE miniflux ist intern ein GRANT miniflux TO <neuer user>, und das darf nur, wer selbst ADMIN OPTION auf miniflux hat. CREATEROLE allein reicht seit PG 16 nicht mehr. Ohne diese Zeile schlägt bao read database/creds/miniflux mit permission denied to grant role "miniflux" fehl.

Kontrolle mit \du:

Role name	Attributes
demo	Create DB
miniflux	Cannot login
openbao	Create role
postgres	Superuser, Create role, Create DB,
...	
v-kubernetes-demo-app-A1EdiNgGWCIkBP74UKVD-1789630487	Password valid until 2026-09-17 08:34:52+00
v-kubernetes-demo-app-DmrcWpu6p4vD4TULtYCp-1789632287	Password valid until 2026-09-17 09:04:52+00

Die v-kubernetes-...-Einträge sind die dynamischen User der anderen Demo-App. Dass mehrere gleichzeitig existieren, ist normal: ESO holt alle 30 Minuten neue, OpenBao löscht sie erst nach Ablauf der Lease (1 h). Zwei bis drei parallele User pro Rolle sind also der erwartete Zustand.

Verifikation: Läuft die Session wirklich als miniflux?

Bevor irgendetwas in Kubernetes angefasst wird, lohnt ein Test von Hand. Credentials holen (mit Root-Token; deshalb heißt der User v-root-...):

```
/ $ bao read database/creds/miniflux
Key          Value
---          -
lease_id     database/creds/miniflux/yVVxUMH7xmpld2lCx0kldNxq
lease_duration 1h
username     v-root-miniflux-WPF9Sj2Qy81wRslpzTRv-1789633845
password     <...>
```

Und damit anmelden:

```
kubectl exec -i -n postgres postgres-postgresql-0 -- sh -c \
'PGPASSWORD="<pw>" psql -U v-root-miniflux-WPF9Sj2Qy81wRslpzTRv-1789633845 -d demodb' <
<'EOF'
SELECT current_user, current_role;
CREATE TABLE _probe(x int);
SELECT tableowner FROM pg_tables WHERE tablename = '_probe';
DROP TABLE _probe;
EOF
```

```
current_user | current_role
-----+-----
miniflux    | miniflux

CREATE TABLE
tableowner
-----
miniflux

DROP TABLE
```

Genau das wollten wir sehen: Die Session ist miniflux, und eine neu angelegte Tabelle gehört miniflux – nicht dem v-root-...-User. Wäre current_role hier der v-...-Name, hätte ALTER ROLE ... SET ROLE nicht gegriffen.

Teil IV – External Secrets Operator anbinden

Drei Objekte, alle im Namespace demo.

SecretStore: Wie ESO zu OpenBao kommt

```
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: openbao
  namespace: demo
spec:
  provider:
    vault:
      server: http://openbao.openbao.svc:8200
      path: secret
      version: v2
      auth:
        kubernetes:
          mountPath: kubernetes
          role: eso-demo
          serviceAccountRef:
            name: demo
```

Der SecretStore wird für den KV-Teil (secret/...) gebraucht. Für dynamische Credentials verwenden wir ihn nicht direkt – der Generator bringt seine eigene Provider-Konfiguration mit. Beide sehen sich aber sehr ähnlich, und der Grund ist derselbe: Sie beschreiben *wie* und *als* wer sich ESO anmeldet.

VaultDynamicSecret: Der Generator

```
apiVersion: generators.external-secrets.io/v1alpha1
kind: VaultDynamicSecret
metadata:
  name: postgres-demo-creds
  namespace: demo
spec:
  path: /database/creds/miniflux
  method: GET
  resultType: Data

  provider:
    server: "http://openbao.openbao.svc:8200"
    auth:
      kubernetes:
        mountPath: kubernetes
        role: eso-demo
        serviceAccountRef:
          name: demo
```

path ist der OpenBao-Pfad, method: GET entspricht bao read, und resultType: Data sagt ESO, dass die Felder unter .data (also username und password) das Ergebnis sind. Bei jedem Sync ruft ESO diesen Pfad neu auf – jedes Mal entsteht ein neuer Datenbank-User.

ExternalSecret: Das Ziel-Secret

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: miniflux-db
  namespace: demo
spec:
  refreshInterval: 30m

  target:
    name: miniflux-db
    creationPolicy: Owner
    template:
      data:
        DATABASE_URL: >-
          postgres://{{ .username }}:{{ .password }}@postgres-postgresql.postgres.svc.cluster.local:5432/demodb?sslmode=disable

  dataFrom:
    - sourceRef:
        generatorRef:
          apiVersion: generators.external-secrets.io/v1alpha1
          kind: VaultDynamicSecret
          name: postgres-demo-creds
```

Zwei Details:

- dataFrom mit generatorRef statt des üblichen secretStoreRef. Das ist die Kopplung an den Generator.
- Das template baut aus username und password eine fertige DATABASE_URL. Miniflux erwartet genau diese eine Variable, und so bleibt das Deployment frei von jeder OpenBao-Kenntnis.

refreshInterval: 30m bei default_ttl: 1h in OpenBao bedeutet: Es gibt immer gültige Credentials im Secret, bevor die alten ablaufen. Halb so lang wie die TTL ist eine gute Faustregel.

Anwenden und prüfen

```
kubectl apply -f miniflux/vault-dynamic-secret.yaml
kubectl apply -f miniflux/miniflux-external-secret.yaml
kubectl get externalsecret -n demo miniflux-db
```

NAME	REFRESH INTERVAL	STATUS	READY	LAST SYNC
miniflux-db	30m	SecretSynced	True	8s

Einen Sync sofort erzwingen (z. B. nach Änderung des Generators) geht über eine beliebige Annotation-Änderung am ExternalSecret:

```
kubectl annotate externalsecret -n demo miniflux-db force-sync="$(date +%s)" --overwrite
```

Und ein Blick in das erzeugte Secret, mit maskiertem Passwort:

```
kubectl get secret -n demo miniflux-db -o jsonpath='{.data.DATABASE_URL}' \
| base64 -d | sed -E 's#://([^:]+):([^\@]+@#://\1:<pw>@#'
```

```
postgres://v-kubernetes-miniflux-SkwRAKNo7i4xy0uFFD63-1789634099:<pw>@postgres-postgresql.po
stgres.svc.cluster.local:5432/demodb?sslmode=disable
```

v-kubernetes-miniflux-... – die Credentials kamen über die Kubernetes-Auth und die Rolle miniflux.
Die Kette steht.

Teil V – Die Anwendung

Das Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: miniflux
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: miniflux
  template:
    metadata:
      labels:
        app: miniflux
    spec:
      containers:
        - name: miniflux
          image: miniflux/miniflux:2.3.3
          ports:
            - containerPort: 8080
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: miniflux-db
                  key: DATABASE_URL
            - name: RUN_MIGRATIONS
              value: "1"
            - name: CREATE_ADMIN
              value: "1"
            - name: ADMIN_USERNAME
              value: "admin"
            - name: ADMIN_PASSWORD
              value: "admin123"
```

Nichts daran ist OpenBao-spezifisch. Das Deployment liest ein ganz normales Kubernetes-Secret. Genau das ist der Punkt: Die Anwendung muss nichts von dynamischen Credentials wissen.

Ein Tipp aus der Praxis: Während der Einrichtung ist es hilfreich, den Container mit `command: ["sh", "-c", "sleep infinity"]` zu starten. Dann kann man per `kubectl exec` hinein, `echo $DATABASE_URL` prüfen und mit dem mitgelieferten Binary (`/usr/bin/miniflux`) manuell starten. Vor dem echten Betrieb muss dieser Block wieder raus – sonst läuft nur `sleep`, und die Anwendung startet nie.

```
kubectl apply -f miniflux/miniflux-deployment.yaml
kubectl rollout status -n demo deploy/miniflux
kubectl logs -n demo deploy/miniflux
```

Ein erfolgreicher Start sieht so aus:

```
level=INFO msg="Running database migrations" current_version=0 latest_version=132  
level=INFO msg="Created new admin user" username=admin user_id=1  
level=INFO msg="Starting HTTP server" listen_address=0.0.0.0:8080
```

Die Oberfläche ansehen

Für den schnellen Blick reicht ein Port-Forward, ohne Service oder Ingress:

```
kubectl port-forward -n demo deploy/miniflux 8080:8080
```

Dann `http://localhost:8080` im Browser, Login `admin / admin123`.

Für dauerhaften Zugang: Ein Service vom Typ ClusterIP plus ein Ingress auf Traefik, nach demselben Muster wie beim OpenBao-Chart.

Teil VI – Was schiefging, und warum

Dieser Teil ist die eigentliche Dokumentation. Jeder der Fehler war eine Schranke, die absichtlich existiert.

permission denied for schema public

```
level=INFO msg="Running database migrations" current_version=0 latest_version=132  
[Migration v1] pq: permission denied for schema public at position 2:17 (42501)
```

Der Pod war im CrashLoopBackOff. Die Verbindung selbst funktionierte – Host, Port, Auth, Datenbank waren alle richtig. Der User durfte nur keine Tabellen anlegen. Ursache: PostgreSQL ≥ 15 gibt PUBLIC kein CREATE mehr auf dem Schema public, und die Rolle demo-app grantete es auch nicht.

Lösung: Owner-Rolle mit CREATE auf dem Schema, dynamische User werden Mitglied (Teil III).

"miniflux" is not an allowed role

HTTP 500 beim Lesen von database/creds/miniflux. Die Rolle existierte, war aber nicht in allowed_roles der Connection. Lösung: Allowlist erweitern (Teil II).

permission denied (403) bei allem

Zwei verschiedene Ursachen mit derselben Meldung:

1. **Kein oder abgelaufener Token im aufrufenden Terminal.** Erkennbar daran, dass auch bao token lookup scheitert. Lösung: bao login.
2. **Policy erlaubt den Pfad nicht.** Erkennbar mit bao token capabilities <pfad> – liefert deny. Lösung: Policy ergänzen (Teil II).

Beim Debuggen von ESO ist die zweite Ursache die häufigere. Die Meldung taucht dann im status des ExternalSecret bzw. in den Logs des ESO-Pods auf, nicht in der Anwendung.

permission denied to grant role "miniflux"

Trat hier *nicht* auf, weil der Grant mit ADMIN OPTION vorher gesetzt war – aber er ist die typische Falle ab PostgreSQL 16 beim Owner-Role-Muster (Teil III).

Logs des falschen Pods

kubectl logs deploy/miniflux wählt bei einem laufenden Rollout gern den alten, terminierenden Pod (Found 2 pods, using pod/...). Im Zweifel den Pod explizit benennen oder kubectl rollout status abwarten.

Teil VII – Betrieb

Die TTL-Falle: Env-Variablen sind statisch

Das ist die wichtigste offene Stelle im aktuellen Aufbau. ESO erneuert das Secret alle 30 Minuten, aber ein Container liest Umgebungsvariablen **nur beim Start**. Nach einer Stunde löscht OpenBao den User, mit dem Miniflux gestartet wurde. Bestehende Verbindungen im Pool laufen weiter, aber jeder neue Verbindungsaufbau schlägt fehl – die Anwendung degradiert schleichend.

Drei Auswege:

- **Restart-Trigger auf Secret-Änderung.** Der Stakater *Reloader* beobachtet Secrets und startet betroffene Deployments neu. Eine Annotation am Deployment genügt:
`reloader.stakater.com/auto: "true"`. Der Neustart alle 30 Minuten ist für Miniflux unproblematisch.
- **Secret als Datei mounten** statt als Env-Variable. Kubernetes aktualisiert gemountete Secrets im laufenden Container. Hilft aber nur, wenn die Anwendung die Datei bei jedem Verbindungsaufbau neu liest – Miniflux tut das nicht.
- **Längere TTL.** `default_ttl=24h` mit `refreshInterval: 12h` reduziert die Häufigkeit, löst das Problem aber nicht.

Für den produktiven Betrieb ist die erste Option der übliche Weg.

Root-Token ablösen

Die Einrichtung lief mit dem Root-Token. Der gehört nach der Ersteinrichtung widerrufen (`bao token revoke <root-token>`) und bei Bedarf über die Unseal-Keys neu erzeugt (`bao operator generate-root`). Für tägliche Administration: eine eigene Policy und z. B. Userpass- oder OIDC-Login.

Infrastructure as Code

Alle bao-Kommandos aus Teil II sind Zustand, der bei einem Neuaufbau verloren geht. Das Repo verwaltet Auth-Methoden, Policies und Audit bereits mit OpenTofu unter `tofu/`. Die hier gezeigten Objekte gehören dorthin:

Objekt	Tofu-Ressource
Database Engine	<code>vault_mount (type = "database")</code>
Connection postgres-demo	<code>vault_database_secret_backend_connection</code>
Rollen demo-app, miniflux	<code>vault_database_secret_backend_role</code>
Policy eso-demo	<code>vault_policy</code>
K8s-Auth-Rolle eso-demo	<code>vault_kubernetes_auth_backend_role</code>

Die SQL-Statements aus Teil III (Rollen openbao, miniflux) sind Datenbank-Zustand und gehören in ein Init-Script oder eine Migration des Postgres-Deployments.

Klartext-Passwörter

ADMIN_PASSWORD: "admin123" im Deployment und alle Passwörter in postgres/values.yaml sind Demo-Kompromisse. Das Miniflux-Admin-Passwort gehört ins KV-Engine (secret/demo/miniflux) und per ExternalSecret ins Deployment – genau wie die DATABASE_URL, nur ohne Generator.

Lease-Hygiene

Wer viel testet, produziert viele Leases. Übersicht und Aufräumen:

```
bao list sys/leases/lookup/database/creds/miniflux
bao lease revoke -prefix database/creds/miniflux    # alle auf einmal
```

Das Revoke führt das DROP ROLE in PostgreSQL aus. Dank Owner-Role-Muster funktioniert es immer, weil die dynamischen User keine Objekte besitzen.

Anhang A – Alle Kommandos auf einen Blick

In der Reihenfolge, in der sie nötig sind. Vorausgesetzt: Cluster, Helm, kubectl-Kontext.

```

# — Cluster —————
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  -n openbao --create-namespace -f helm/values.yaml
kubectl exec -n openbao openbao-0 -- bao operator init
kubectl exec -n openbao openbao-0 -- bao operator unseal # x3

helm upgrade --install postgres oci://registry-1.docker.io/bitnamicharts/postgresql \
  -n postgres --create-namespace -f postgres/values.yaml

helm upgrade --install external-secrets external-secrets/external-secrets \
  --version 2.10.0 -n external-secrets --create-namespace

kubectl create namespace demo
kubectl create serviceaccount demo -n demo

# — PostgreSQL (als postgres in demodb) —————
CREATE ROLE openbao WITH LOGIN PASSWORD '...' CREATEROLE;
CREATE ROLE miniflux NOLOGIN;
GRANT CONNECT ON DATABASE demodb TO miniflux;
GRANT USAGE, CREATE ON SCHEMA public TO miniflux;
GRANT miniflux TO openbao WITH ADMIN OPTION;

# — OpenBao (Root-Token, Shell im Pod) —————
bao auth enable kubernetes
bao write auth/kubernetes/config kubernetes_host="https://kubernetes.default.svc:443"
bao write auth/kubernetes/role/eso-demo \
  bound_service_account_names=demo bound_service_account_namespaces=demo \
  token_policies=eso-demo token_ttl=1h

bao secrets enable database
bao write database/config/postgres-demo \
  plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@postgres-postgresql.postgres.svc.
    cluster.local:5432/demodb?sslmode=disable" \
  username=openbao password='...' password_authentication=scram-sha-256 \
  allowed_roles="demo-app,miniflux"

bao write database/roles/miniflux db_name=postgres-demo default_ttl=1h max_ttl=24h \
  creation_statements='CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '""'{{password}}'""' V
    ALID UNTIL '""'{{expiration}}'""' IN ROLE miniflux; ALTER ROLE "{{name}}" SET R
    OLE miniflux;'

bao policy write eso-demo - <<'EOF'
path "secret/data/demo/demo-app" { capabilities = ["read"] }
path "secret/metadata/demo/demo-app" { capabilities = ["read"] }
path "database/creds/demo-app" { capabilities = ["read"] }
path "database/creds/miniflux" { capabilities = ["read"] }
EOF

bao read database/creds/miniflux # Test

# — ESO + App —————
kubectl apply -f miniflux/vault-dynamic-secret.yaml
kubectl apply -f miniflux/miniflux-external-secret.yaml
kubectl apply -f miniflux/miniflux-deployment.yaml
kubectl get externalsecret -n demo miniflux-db
kubectl logs -n demo deploy/miniflux
kubectl port-forward -n demo deploy/miniflux 8080:8080

```

Anhang B – Glossar

Auth-Methode – Wie sich ein Client bei OpenBao ausweist. Hier: kubernetes (ServiceAccount-Token) und token (Root-Token).

Policy – Regelwerk, welche Pfade ein Token mit welchen Capabilities (read, create, update, delete, list) benutzen darf. Default: alles verboten.

Secrets Engine – Ein Plugin, das Secrets bereitstellt. kv speichert statische Werte, database erzeugt dynamische Credentials.

Connection – Im Database Engine: die Verbindung zu einer konkreten Datenbank inklusive Allowlist der Rollen (allowed_roles).

Rolle (OpenBao) – Im Database Engine: die SQL-Vorlage zur Erzeugung eines Users plus TTL. Nicht zu verwechseln mit einer PostgreSQL-Rolle.

Rolle (PostgreSQL) – Ein Benutzer oder eine Gruppe. LOGIN macht sie zum Benutzer, NOLOGIN zur reinen Gruppe. Mitgliedschaft per GRANT rolle TO user oder CREATE ROLE ... IN ROLE rolle.

Lease – Die Lebensdauer eines dynamischen Secrets. Läuft sie ab, führt OpenBao die Revocation aus (hier: DROP ROLE).

ADMIN OPTION – Erlaubt einer Rolle, andere zu Mitgliedern einer Rolle zu machen. Seit PostgreSQL 16 Voraussetzung für IN ROLE durch Nicht-Superuser.

SET ROLE – Wechselt innerhalb einer Session die effektive Rolle. ALTER ROLE u SET ROLE r macht das zum Default beim Verbindungsaufbau.

Generator (ESO) – Ein ESO-Objekt, das bei jedem Sync ein neues Secret *erzeugt* statt eines zu lesen. VaultDynamicSecret ruft dafür einen OpenBao-Pfad auf.

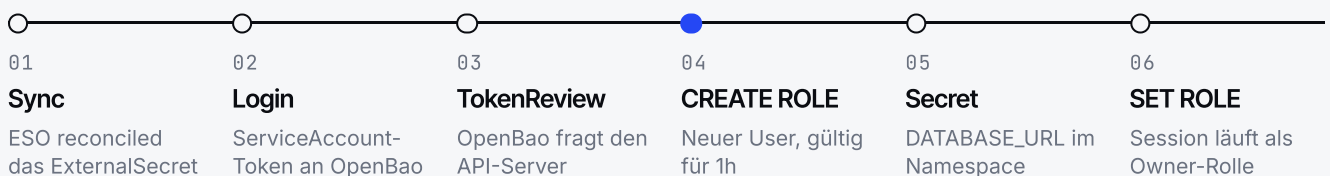
TokenReview – Kubernetes-API, mit der OpenBao prüft, ob ein ServiceAccount-Token echt und gültig ist.

Die Anwendung bekommt kein festes Passwort mehr. Sie bekommt einen Benutzer, der in einer Stunde nicht mehr existiert.

Dieser Leitfaden dokumentiert einen vollständigen, real durchgeführten Aufbau: OpenBao erzeugt auf Anfrage PostgreSQL-Benutzer mit kurzer Lebensdauer, der External Secrets Operator legt sie als Kubernetes-Secret ab, und Miniflux liest sie wie jede andere Umgebungsvariable – ohne je von OpenBao zu wissen.

Alle Kommandos, alle Manifeste, alle SQL-Statements. Und alle Fehler, die unterwegs passiert sind: permission denied for schema public, is not an allowed role, das 403, das nur ein fehlendes Login war. Jeder davon ist eine Schranke, die absichtlich existiert.

```
-- Der Kern des Leitfadens in zwei Zeilen
CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}'
  VALID UNTIL '{{expiration}}' IN ROLE miniflux;
ALTER ROLE "{{name}}" SET ROLE miniflux;
```



kubernetes	RKE2 · Traefik · Longhorn
openbao	2.6.2 · chart 0.29.4 · raft
postgresql	18.6 · bitnami chart 18.11.3
external-secrets	2.10.0 · VaultDynamicSecret
miniflux	2.3.3

28 Seiten · 7 Teile · 2 Anhänge
Vertiefung: **Vault in Practice**
leanpub.com/vault-in-practice
Gesetzt in Inter Tight, Inter und JetBrains Mono.
Alle Ausgaben sind echte Terminal-Ausgaben.